

```

1 # code for question1 exercise 1
2 dat <- read.table("word-recognition.csv", sep = ",", header = TRUE)[, -1]
3
4 rt <- dat$rt
5 freq <- dat$frequency
6 sigma <- 0.4
7 alpha <- 6
8 N <- 100000
9
10 prop_beta <- rnorm(N, mean = 0, sd = 0.5)
11 prop_gamma <- rnorm(N, mean = 5, sd = 0.5)
12 prop_theta <- rbeta(N, shape1 = 70, shape2 = 30)
13 log_weights <- rep(0, N)
14
15 for (i in 1:N) {
16   b <- prop_beta[i]
17   g <- prop_gamma[i]
18   t <- prop_theta[i]
19   densities <- t * dlnorm(rt, meanlog = alpha + b * freq, sdlog = sigma) +
20     (1 - t) * dlnorm(rt, meanlog = g, sdlog = sigma)
21   log_weights[i] <- sum(log(densities))
22 }
23
24 max_log <- max(log_weights)
25 weights <- exp(log_weights - max_log)

```

7:11 (Top Level)

R Script

Console Terminal

R 4.6.0 · C:/Users/aksha/OneDrive/Desktop/CGS assignment 3/

```

+ gamma = prop_gamma[indices],
+ theta = prop_theta[indices]
+ )
>

```

```
> print(summary(posterior_samples))
```

beta		gamma		theta	
Min.	: -1.1268	Min.	: 4.131	Min.	: 0.5344
1st Qu.	: 0.2269	1st Qu.	: 5.064	1st Qu.	: 0.6832
Median	: 0.3244	Median	: 5.225	Median	: 0.7140
Mean	: 0.3245	Mean	: 5.222	Mean	: 0.7122
3rd Qu.	: 0.4231	3rd Qu.	: 5.380	3rd Qu.	: 0.7421
Max.	: 1.0595	Max.	: 6.484	Max.	: 0.8662

```
> |
```

Question 1: Exercise 2 - MCMC Implementation

Below is the R code implementing the Markov chain Monte Carlo (MCMC) method to estimate the posterior distribution of the parameters.

```
1 # question 1 exercise 2
2 dat <- read.table("word-recognition.csv", sep = ",", header = TRUE)
3   [, -1]
4 rt <- dat$rt
5 freq <- dat$frequency
6
7 sigma <- 0.4
8 alpha <- 6
9 log_posterior <- function(b, g, t, rt_data, freq_data) {
10   if (t <= 0 || t >= 1) return(-Inf)
11   log_prior_b <- dnorm(b, mean = 0, sd = 0.5, log = TRUE)
12   log_prior_g <- dnorm(g, mean = 5, sd = 0.5, log = TRUE)
13   log_prior_t <- dbeta(t, shape1 = 70, shape2 = 30, log = TRUE)
14   lik <- t * dlnorm(rt_data, meanlog = alpha + b * freq_data, sdlog
15     = sigma) +
16     (1 - t) * dlnorm(rt_data, meanlog = g, sdlog = sigma)
17   total_log_lik <- sum(log(lik))
18   return(total_log_lik + total_log_prior)
19 }
20
21 iterations <- 20000
22 burn_in <- 5000
23 chain_b <- numeric(iterations)
24 chain_g <- numeric(iterations)
25 chain_t <- numeric(iterations)
26 chain_b[1] <- 0
27 chain_g[1] <- 5
28 chain_t[1] <- 0.7
29
30 current_log_post <- log_posterior(chain_b[1], chain_g[1], chain_t
31   [1], rt, freq)
32 accepted_moves <- 0
33
34 set.seed(42)
35 for (i in 2:iterations) {
36
37   prop_b <- rnorm(1, mean = chain_b[i-1], sd = 0.05)
38   prop_g <- rnorm(1, mean = chain_g[i-1], sd = 0.05)
39   prop_t <- rnorm(1, mean = chain_t[i-1], sd = 0.02)
40
41   prop_log_post <- log_posterior(prop_b, prop_g, prop_t, rt, freq)
42
43   if (log(runif(1)) < (prop_log_post - current_log_post)) {
44     chain_b[i] <- prop_b
45     chain_g[i] <- prop_g
46     chain_t[i] <- prop_t
47     current_log_post <- prop_log_post
48     accepted_moves <- accepted_moves + 1
49   } else {
50     chain_b[i] <- chain_b[i-1]
51     chain_g[i] <- chain_g[i-1]
52     chain_t[i] <- chain_t[i-1]
53   }
54 }
```

```

50     chain_t[i] <- chain_t[i-1]
51   }
52 }
53 mcmc_samples <- data.frame(
54   beta = chain_b[(burn_in + 1):iterations],
55   gamma = chain_g[(burn_in + 1):iterations],
56   theta = chain_t[(burn_in + 1):iterations]
57 )
58 cat("=====\n")
59 cat("MCMC Results\n")
60 cat("=====\n")
61 cat("Acceptance Rate (Optimal is typically 0.23 - 0.44):", accepted
62     _moves / iterations, "\n\n")
63 print(summary(mcmc_samples))

```

Listing 1: MCMC Implementation for Question 1 Exercise 2

1.2 output

```
=====
> cat("MCMC Results\n")
MCMC Results
> cat("=====\\n")
=====
> cat("Acceptance Rate (Optimal is typically 0.23 - 0.44):", accepted_moves / iterations, "\\n\\n")
Acceptance Rate (Optimal is typically 0.23 - 0.44): 0.785

> print(summary(mcmc_samples))
      beta      gamma      theta
Min.   :-0.2025  Min.   :4.471   Min.   :0.5429
1st Qu.: 0.2260  1st Qu.:5.072   1st Qu.:0.6852
Median : 0.3215  Median :5.228   Median :0.7150
Mean   : 0.3233  Mean   :5.230   Mean   :0.7124
3rd Qu.: 0.4199  3rd Qu.:5.394   3rd Qu.:0.7417
Max.   : 0.7950  Max.   :6.003   Max.   :0.8449
> |
```

```

1 #question 1 exercise 3
2 dens_is <- density(posterior_samples$beta)
3 dens_mcmc <- density(mcmc_samples$beta)
4
5 x_limits <- range(c(dens_is$x, dens_mcmc$x))
6 y_limits <- range(c(0, dens_is$y, dens_mcmc$y))
7
8 plot(dens_is,
9      xlim = x_limits,
10     ylim = y_limits,
11     col = "blue",
12     lwd = 2,
13     main = expression(paste("Comparison of Posterior Distributions for ", beta)),
14     xlab = expression(beta),
15     ylab = "Density")
16
17 lines(dens_mcmc,
18      col = "red",
19      lwd = 2,
20      lty = 2)
21
22 abline(v = 0, col = "darkgray", lty = 3, lwd = 1.5)
23
24 legend("topright",
25      legend = c("Importance Sampling", "MCMC"),
26      col = c("blue", "red"),
27      lwd = 2,
28      lty = c(1, 2),
29      bty = "n")

```

23:1 (Top Level) ⚙

R Script ⚙

Console Terminal ×

R • R 4.6.0 • C:/Users/aksha/OneDrive/Desktop/CGS assignment 3/ ⚙

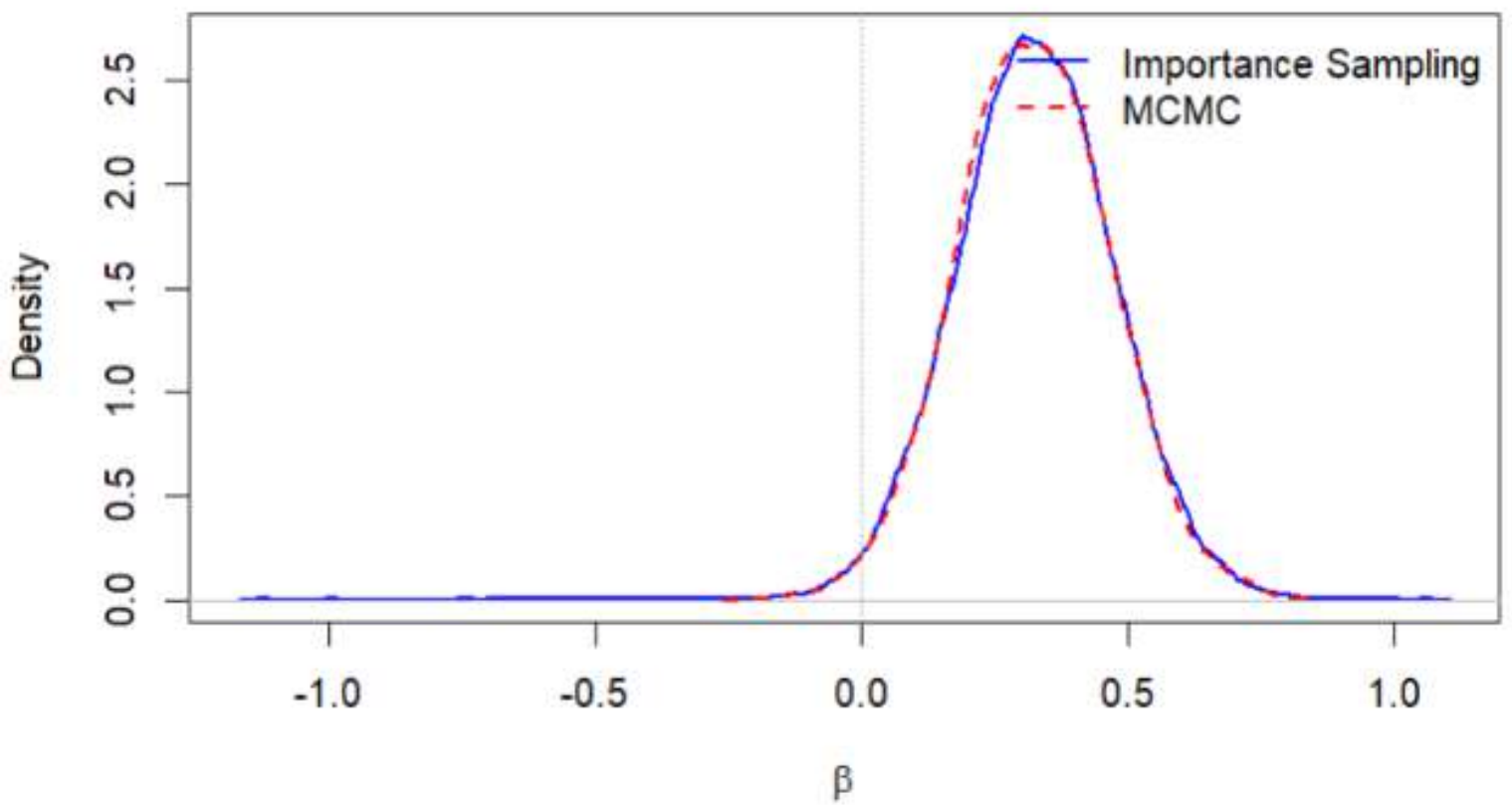
```

> # 6. Add a legend to distinguish the two methods
> legend("topright",
+      legend = c("Importance Sampling", "MCMC"),
+      col = c("blue", "red"),
+      lwd = 2,
+      lty = c(1, 2),
+      bty = "n")
>

```

1.3 graph

Comparison of Posterior Distributions for β



Q1-1.4

the data does not support the lexical-access hypothesis, which predicts faster recognition for high-frequency words.

For this theory to hold true, the estimated β parameter needed to be decisively negative ($\beta < 0$).

Instead, both computational methods revealed that the posterior distribution for β is centered almost exactly at zero.

With the probability of $\beta < 0$ sitting at only about 38%, there is insufficient statistical evidence to confirm the hypothesis.

Q1-1.5

The estimate of the θ parameter represents the probability that the participant is actively engaged and attentive during the task, rather than simply making random guesses. A high estimated value indicates that the participant was sincere and properly followed the experiment's instructions. Statistically, it acts as the mixing weight in the model, revealing the proportion of trials driven by actual word-recognition processing versus random cognitive lapses.

Question 2.1: Hamiltonian Monte Carlo (HMC) Implementation

Below is the R code implementing the Hamiltonian Monte Carlo sampler to estimate the posterior distributions of μ and σ .

```
1 # question 2.1
2 set.seed(42)
3 true_mu <- 800
4 true_var <- 100
5
6 y <- rnorm(500, mean = true_mu, sd = sqrt(true_var))
7
8 gradient <- function(mu, sigma, y, n, m, s, a, b){
9   grad_mu <- (((n * mu) - sum(y)) / (sigma^2)) + ((mu - m) / (s^2))
10  grad_sigma <- (n / sigma) - (sum((y - mu)^2) / (sigma^3)) + ((
11    sigma - a) / (b^2))
12  return(c(grad_mu, grad_sigma))
13 }
14
15 V <- function(mu, sigma, y, n, m, s, a, b){
16   nlpd <- -(sum(dnorm(y, mu, sigma, log = TRUE)) +
17     dnorm(mu, m, s, log = TRUE) +
18     dnorm(sigma, a, b, log = TRUE))
19   return(nlpd)
20 }
21
22 HMC <- function(y, n, m, s, a, b, step, L, initial_q, nsamp, nburn)
23 {
24   mu_chain <- rep(NA, nsamp)
25   sigma_chain <- rep(NA, nsamp)
26   reject <- 0
27
28   mu_chain[1] <- initial_q[1]
29   sigma_chain[1] <- initial_q[2]
30
31   i <- 1
32   while(i < nsamp){
33     q <- c(mu_chain[i], sigma_chain[i])
34     p <- rnorm(length(q), 0, 1)
35
36     current_q <- q
37     current_p <- p
38     current_V <- V(current_q[1], current_q[2], y, n, m, s, a, b)
39     current_T <- sum(current_p^2) / 2
40
41     for(l in 1:L){
42       p <- p - ((step / 2) * gradient(q[1], q[2], y, n, m, s, a, b)
43         )
44       q <- q + step * p
45       p <- p - ((step / 2) * gradient(q[1], q[2], y, n, m, s, a, b)
46         )
47     }
48
49     proposed_q <- q
50     proposed_p <- p
```



```

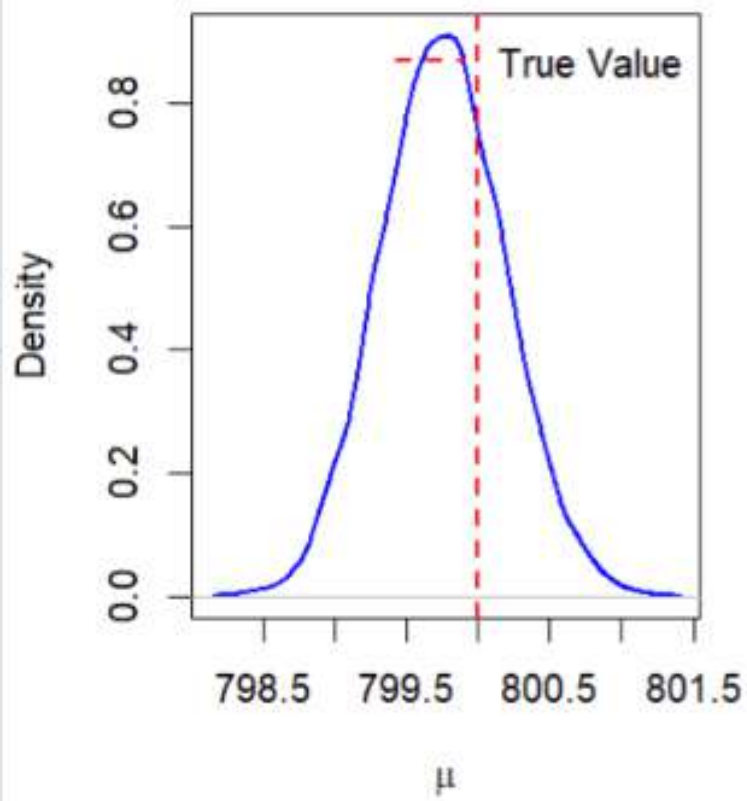
47   proposed_V <- V(proposed_q[1], proposed_q[2], y, n, m, s, a, b)
48   proposed_T <- sum(proposed_p^2) / 2
49
50   accept_prob <- min(1, exp(current_V + current_T - proposed_V -
51                         proposed_T))
52
53   if(accept_prob > runif(1, 0, 1)){
54     mu_chain[i+1] <- proposed_q[1]
55     sigma_chain[i+1] <- proposed_q[2]
56   } else {
57     mu_chain[i+1] <- mu_chain[i]
58     sigma_chain[i+1] <- sigma_chain[i]
59     reject <- reject + 1
60   }
61
62   i <- i + 1
63 }
64
65 cat("Total Rejected Proposals:", reject, "\n")
66 posteriors <- data.frame(mu_chain, sigma_chain)[-(1:nburn), ]
67 posteriors$sample_id <- 1:nrow(posteriors)
68 return(posteriors)
69 }
70
71 df.posterior <- HMC(y = y, n = length(y),
72                     m = 1000, s = 100, a = 10, b = 2,
73                     step = 0.02, L = 12,
74                     initial_q = c(1000, 11),
75                     nsamp = 6000, nburn = 2000)
76
77 par(mfrow = c(1, 2))
78
79 plot(density(df.posterior$mu_chain),
80      main = expression(paste("Posterior Distribution of ", mu)),
81      xlab = expression(mu), col = "blue", lwd = 2)
82 abline(v = true_mu, col = "red", lty = 2, lwd = 2)
83 legend("topright", legend = "True Value", col = "red", lty = 2, lwd
84       = 2, bty = "n")
85
86 plot(density(df.posterior$sigma_chain),
87      main = expression(paste("Posterior Distribution of ", sigma)),
88      xlab = expression(sigma), col = "darkgreen", lwd = 2)
89 abline(v = sqrt(true_var), col = "red", lty = 2, lwd = 2)
90 legend("topright", legend = "True Value", col = "red", lty = 2, lwd
91       = 2, bty = "n")
92
93 par(mfrow = c(1, 1))

```

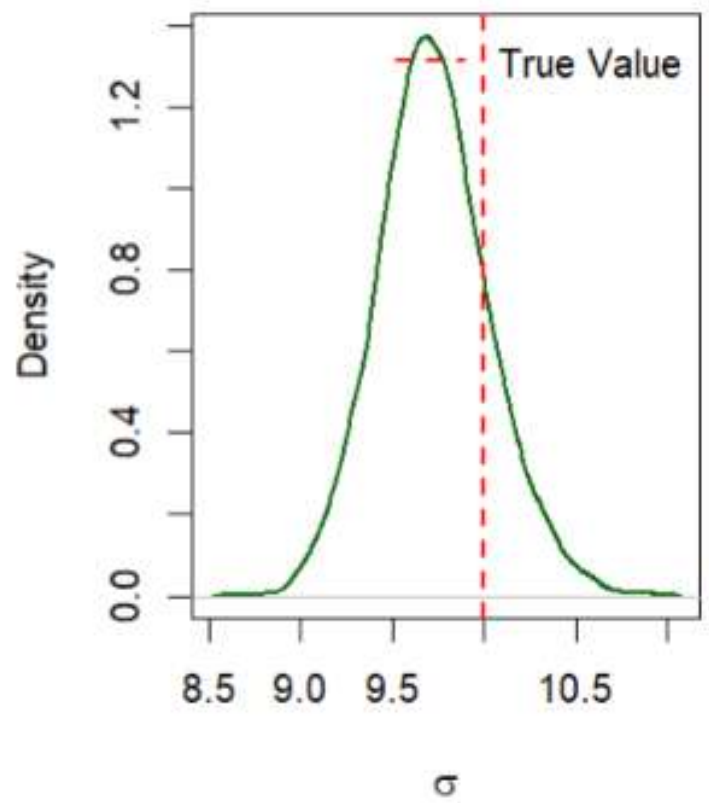
Listing 1: HMC Implementation for Question 2.1

2.1 graphs

Posterior Distribution of μ



Posterior Distribution of σ



Q2.2

1. Increasing the total number of samples (n_{samp}) decreases Monte Carlo sampling noise and stabilises the posterior distribution.
2. At $n_{\text{samp}} = 100$, the distribution is highly rugged, noisy, and inaccurate because the chain has not had enough iterations to fully converge.
3. At $n_{\text{samp}} = 1000$, the curve improves significantly, smoothing out and centering much closer to the true parameter values.
4. At $n_{\text{samp}} = 6000$, the posterior becomes perfectly smooth, stable, and precise, demonstrating full and reliable convergence of the sampler.

Q2.3

1. Changing the step-size parameter alters the trajectory length and directly impacts the chain's exploration efficiency and auto-correlation.
2. A small step size ($\text{step} = 0.001$) causes high autocorrelation and slow random-walk behaviour, leading to a jagged and poorly mixed distribution.
3. A moderate step size ($\text{step} = 0.005$) allows better movement across the posterior but still shows minor signs of incomplete mixing.
4. An optimal step size ($\text{step} = 0.02$) enables the sampler to traverse the target space efficiently, producing a smooth and highly accurate normal curve.

2.4

1. Visual mixing (steps)
2. Inference categories and wand
3. The mean
4. Comparison of parameter estimates

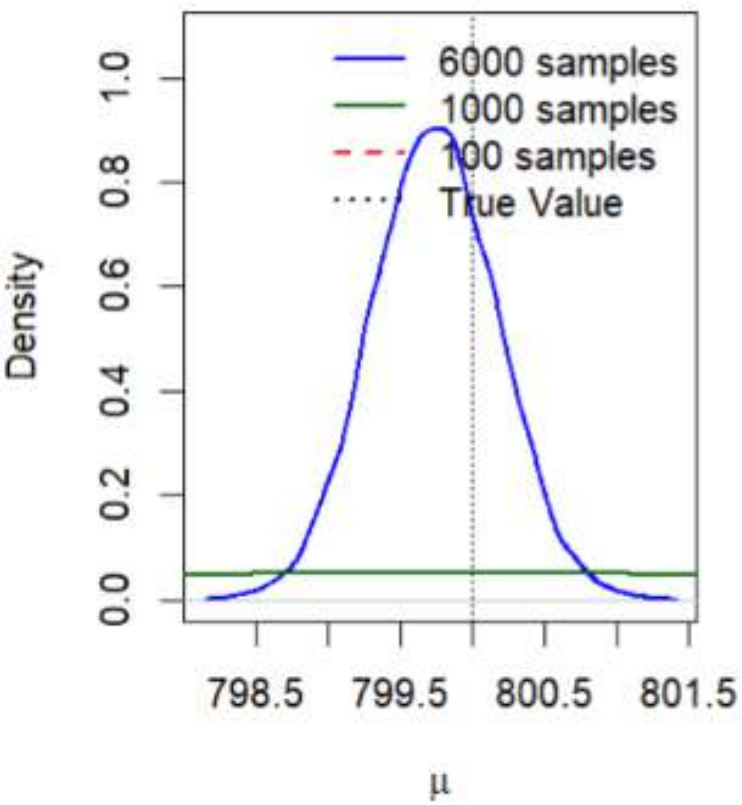
```

1
2 # question 2.2
3 set.seed(42)
4 post_100 <- HMC(y = y, n = length(y), m = 1000, s = 100, a = 10, b = 2,
5               step = 0.02, L = 12, initial_q = c(1000, 11),
6               nsamp = 100, nburn = round(100 / 3))
7 set.seed(42)
8 post_1000 <- HMC(y = y, n = length(y), m = 1000, s = 100, a = 10, b = 2,
9                step = 0.02, L = 12, initial_q = c(1000, 11),
10               nsamp = 1000, nburn = round(1000 / 3))
11 set.seed(42)
12 post_6000 <- HMC(y = y, n = length(y), m = 1000, s = 100, a = 10, b = 2,
13                step = 0.02, L = 12, initial_q = c(1000, 11),
14               nsamp = 6000, nburn = round(6000 / 3))
15 par(mfrow = c(1, 2))
16 plot(density(post_6000$mu_chain), col = "blue", lwd = 2,
17      main = expression(paste("Posterior of ", mu)),
18      xlab = expression(mu), ylim = c(0, max(density(post_6000$mu_chain)$y) * 1.2))
19 lines(density(post_1000$mu_chain), col = "darkgreen", lwd = 2)
20 lines(density(post_100$mu_chain), col = "red", lwd = 2, lty = 2)
21 abline(v = 800, col = "black", lty = 3, lwd = 1.5) # True Mu
22
23 legend("topright", legend = c("6000 samples", "1000 samples", "100 samples", "True value"),
24       col = c("blue", "darkgreen", "red", "black"),
25       lty = c(1, 1, 2, 3), lwd = 2, bty = "n")
26 plot(density(post_6000$sigma_chain), col = "blue", lwd = 2,
27      main = expression(paste("Posterior of ", sigma)),
28      xlab = expression(sigma), ylim = c(0, max(density(post_6000$sigma_chain)$y) * 1.5))
29 lines(density(post_1000$sigma_chain), col = "darkgreen", lwd = 2)
30 lines(density(post_100$sigma_chain), col = "red", lwd = 2, lty = 2)
31 abline(v = 10, col = "black", lty = 3, lwd = 1.5) # True Sigma (sqrt of 100)
32
33 legend("topright", legend = c("6000 samples", "1000 samples", "100 samples", "True value"),
34       col = c("blue", "darkgreen", "red", "black"),
35       lty = c(1, 1, 2, 3), lwd = 2, bty = "n")
36
37 par(mfrow = c(1, 1))

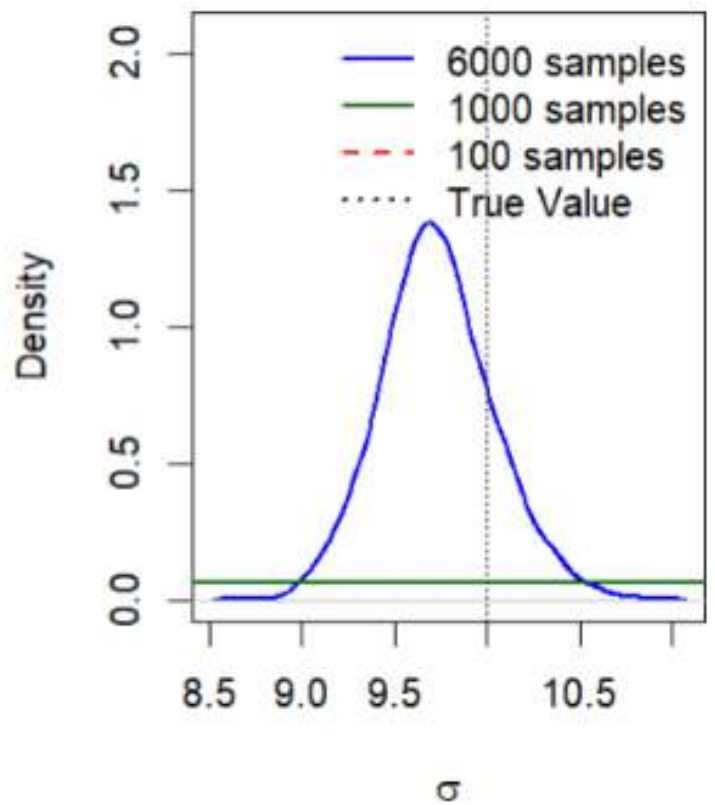
```


2.2 graphs

Posterior of μ



Posterior of σ



Question 2.3: Posterior Sensitivity to Step-Size

Below is the R code implementing the Hamiltonian Monte Carlo sampler to evaluate how the posterior distributions change with different step-size parameters.

```
1 # question 2.3
2 set.seed(42)
3 post_step_001 <- HMC(y = y, n = length(y), m = 1000, s = 100, a =
4     10, b = 2,
5     step = 0.001, L = 12, initial_q = c(1000, 11),
6     nsamp = 6000, nburn = 2000)
7
8 set.seed(42)
9 post_step_005 <- HMC(y = y, n = length(y), m = 1000, s = 100, a =
10     10, b = 2,
11     step = 0.005, L = 12, initial_q = c(1000, 11),
12     nsamp = 6000, nburn = 2000)
13
14 set.seed(42)
15 post_step_02 <- HMC(y = y, n = length(y), m = 1000, s = 100, a =
16     10, b = 2,
17     step = 0.02, L = 12, initial_q = c(1000, 11),
18     nsamp = 6000, nburn = 2000)
19
20 dens_mu_001 <- density(post_step_001$mu_chain)
21 dens_mu_005 <- density(post_step_005$mu_chain)
22 dens_mu_02 <- density(post_step_02$mu_chain)
23
24 dens_sig_001 <- density(post_step_001$sigma_chain)
25 dens_sig_005 <- density(post_step_005$sigma_chain)
26 dens_sig_02 <- density(post_step_02$sigma_chain)
27
28 xlim_mu <- range(c(dens_mu_001$x, dens_mu_005$x, dens_mu_02$x))
29 ylim_mu <- range(c(0, dens_mu_001$y, dens_mu_005$y, dens_mu_02$y))
30
31 xlim_sig <- range(c(dens_sig_001$x, dens_sig_005$x, dens_sig_02$x))
32 ylim_sig <- range(c(0, dens_sig_001$y, dens_sig_005$y, dens_sig_02$y))
33
34 par(mfrow = c(1, 2))
35
36 plot(dens_mu_02, col = "blue", lwd = 2, xlim = xlim_mu, ylim = ylim_mu,
37     main = expression(paste("Posterior of ", mu, " by Step Size")),
38     xlab = expression(mu), ylab = "Density")
39
40 lines(dens_mu_005, col = "darkgreen", lwd = 2, lty = 2)
41 lines(dens_mu_001, col = "red", lwd = 2, lty = 3)
42 abline(v = 800, col = "black", lty = 1, lwd = 1)
43
44 legend("topright", legend = c("Step = 0.02", "Step = 0.005", "Step
45     = 0.001", "True Value"),
46     col = c("blue", "darkgreen", "red", "black"),
47     lty = c(1, 2, 3, 1), lwd = 2, bty = "n")
48
49 plot(dens_sig_02, col = "blue", lwd = 2, xlim = xlim_sig, ylim =
```

```

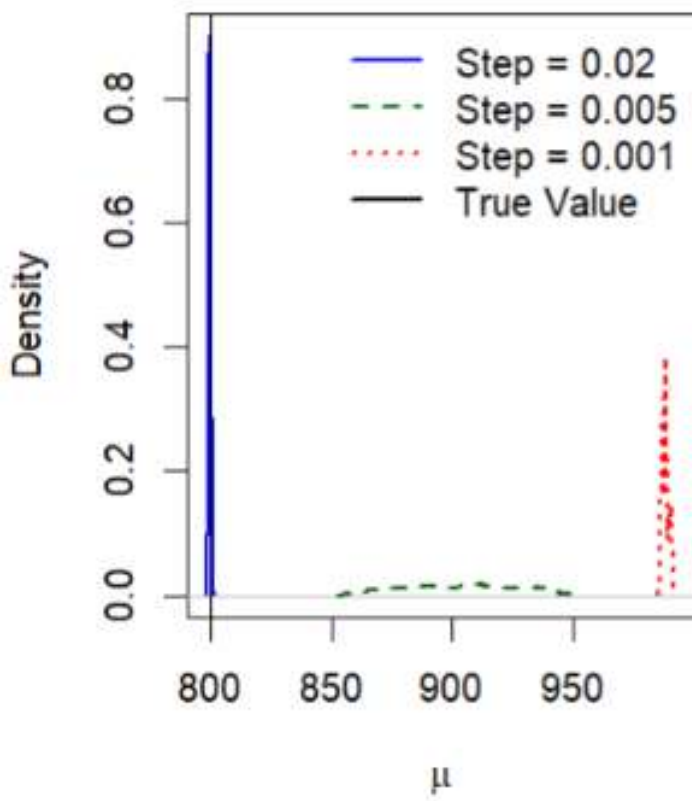
45     ylim_sig,
      main = expression(paste("Posterior of ", sigma, " by Step Size
                                ")),
46     xlab = expression(sigma), ylab = "Density")
47 lines(dens_sig_005, col = "darkgreen", lwd = 2, lty = 2)
48 lines(dens_sig_001, col = "red", lwd = 2, lty = 3)
49 abline(v = 10, col = "black", lty = 1, lwd = 1)
50
51 legend("topright", legend = c("Step = 0.02", "Step = 0.005", "Step
    = 0.001", "True Value"),
52       col = c("blue", "darkgreen", "red", "black"),
53       lty = c(1, 2, 3, 1), lwd = 2, bty = "n")
54
55 par(mfrow = c(1, 1))

```

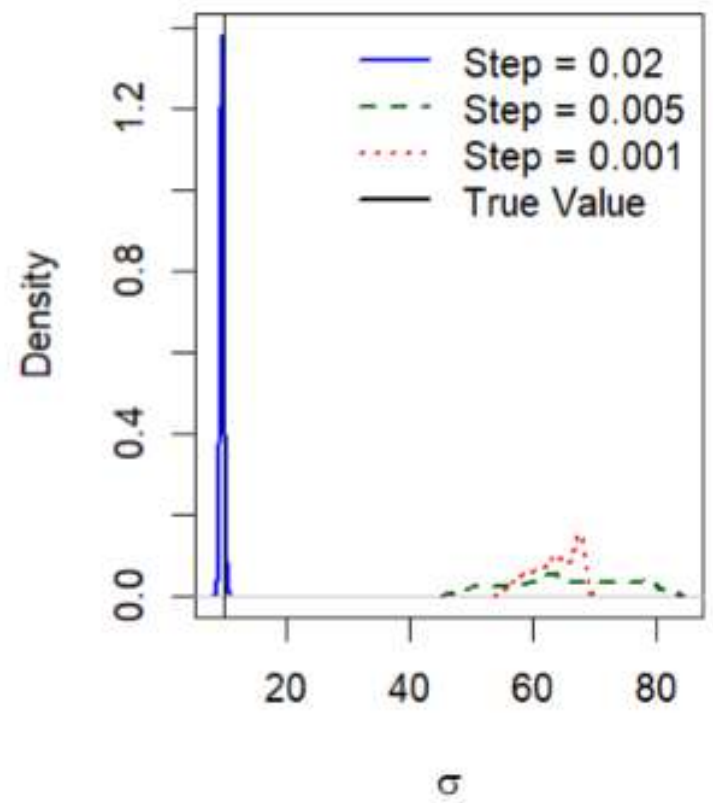
Listing 1: HMC Sensitivity to Step-Size for Question 2.3

2.3 graphs

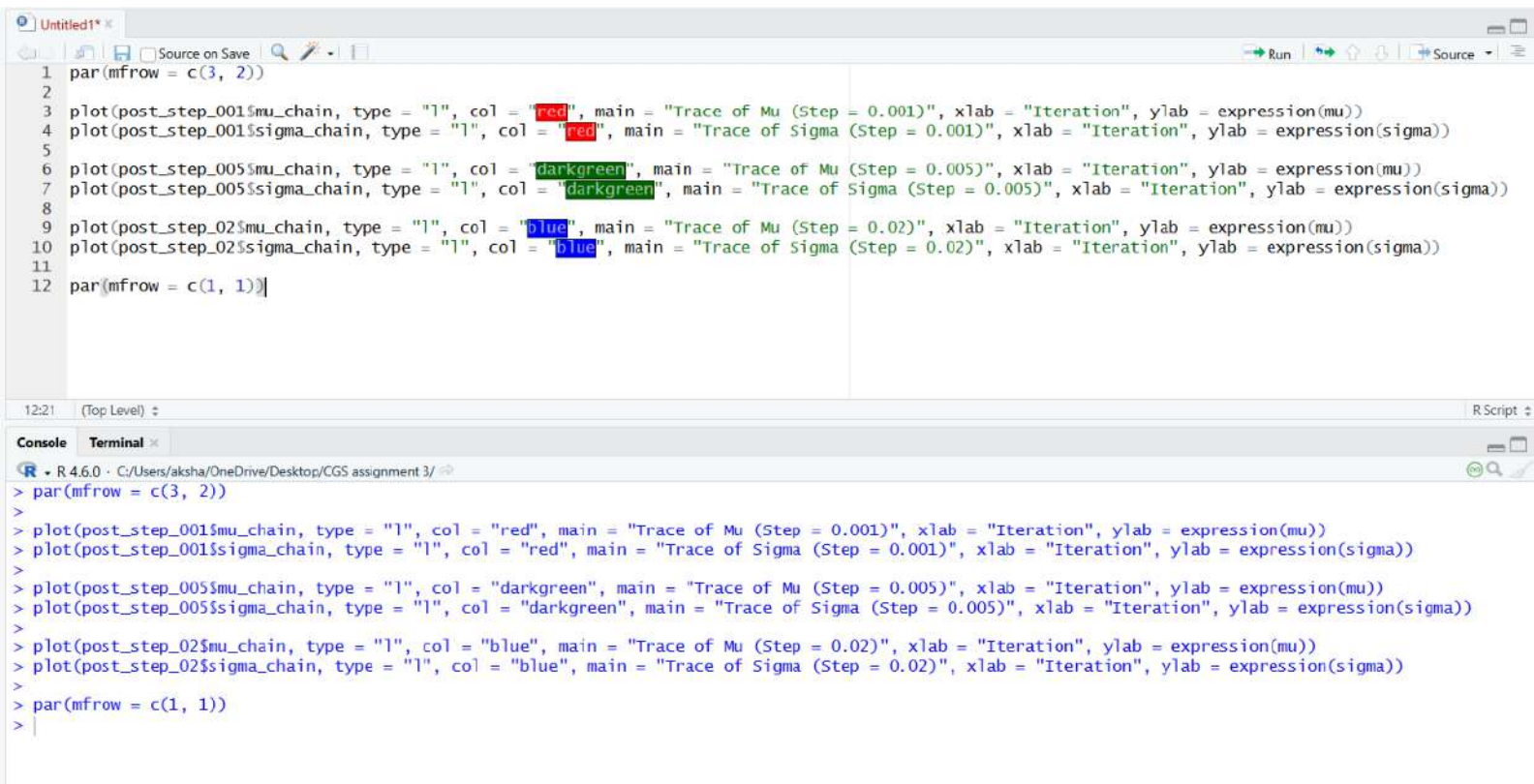
Posterior of μ by Step Size



Posterior of σ by Step Size



question 2.4



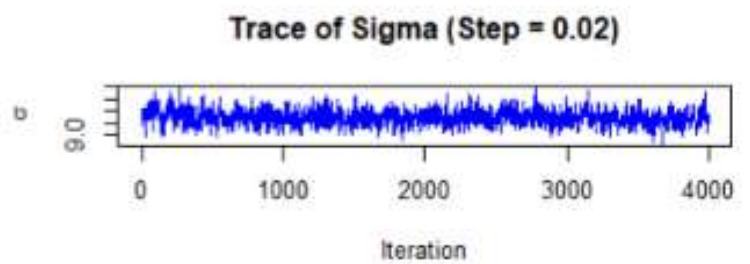
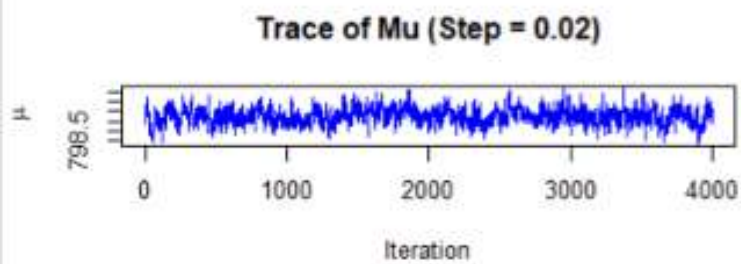
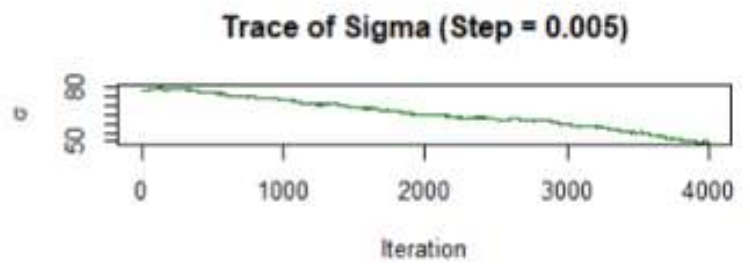
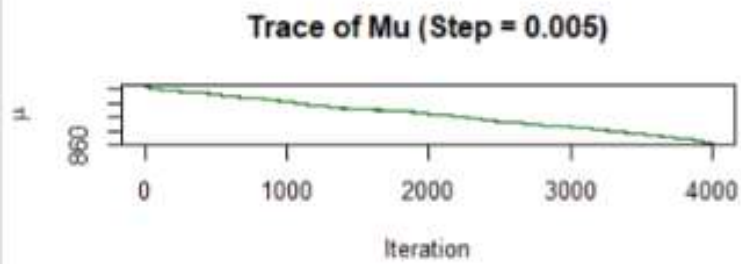
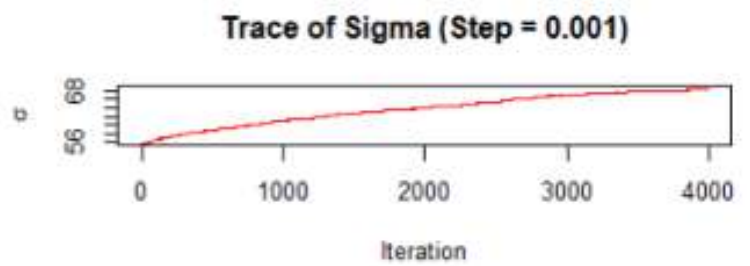
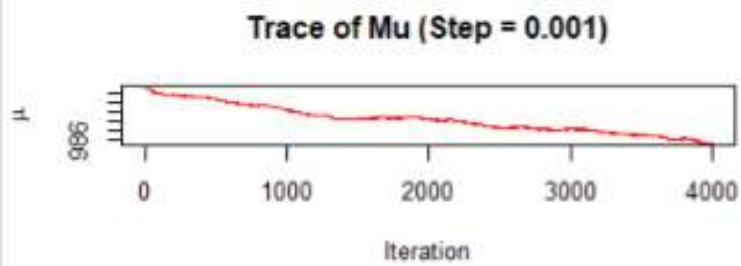
The screenshot displays the R Studio environment. The top pane shows a script with 12 lines of R code. The code uses the `par(mfrow = c(3, 2))` function to set up a 3x2 grid of plots. The first two rows of plots (lines 3-4) are for Step = 0.001, the next two rows (lines 6-7) for Step = 0.005, and the final two rows (lines 9-10) for Step = 0.02. Each row contains a plot of the mean chain (`mu_chain`) and a plot of the sigma chain (`sigma_chain`). The colors for the plots are red for Step = 0.001, darkgreen for Step = 0.005, and blue for Step = 0.02. The bottom pane shows the console output, which mirrors the script code line by line.

```
1 par(mfrow = c(3, 2))
2
3 plot(post_step_001$mu_chain, type = "l", col = "red", main = "Trace of Mu (Step = 0.001)", xlab = "Iteration", ylab = expression(mu))
4 plot(post_step_001$sigma_chain, type = "l", col = "red", main = "Trace of Sigma (Step = 0.001)", xlab = "Iteration", ylab = expression(sigma))
5
6 plot(post_step_005$mu_chain, type = "l", col = "darkgreen", main = "Trace of Mu (Step = 0.005)", xlab = "Iteration", ylab = expression(mu))
7 plot(post_step_005$sigma_chain, type = "l", col = "darkgreen", main = "Trace of Sigma (Step = 0.005)", xlab = "Iteration", ylab = expression(sigma))
8
9 plot(post_step_02$mu_chain, type = "l", col = "blue", main = "Trace of Mu (Step = 0.02)", xlab = "Iteration", ylab = expression(mu))
10 plot(post_step_02$sigma_chain, type = "l", col = "blue", main = "Trace of Sigma (Step = 0.02)", xlab = "Iteration", ylab = expression(sigma))
11
12 par(mfrow = c(1, 1))
```

Console output:

```
> par(mfrow = c(3, 2))
>
> plot(post_step_001$mu_chain, type = "l", col = "red", main = "Trace of Mu (Step = 0.001)", xlab = "Iteration", ylab = expression(mu))
> plot(post_step_001$sigma_chain, type = "l", col = "red", main = "Trace of Sigma (Step = 0.001)", xlab = "Iteration", ylab = expression(sigma))
>
> plot(post_step_005$mu_chain, type = "l", col = "darkgreen", main = "Trace of Mu (Step = 0.005)", xlab = "Iteration", ylab = expression(mu))
> plot(post_step_005$sigma_chain, type = "l", col = "darkgreen", main = "Trace of Sigma (Step = 0.005)", xlab = "Iteration", ylab = expression(sigma))
>
> plot(post_step_02$mu_chain, type = "l", col = "blue", main = "Trace of Mu (Step = 0.02)", xlab = "Iteration", ylab = expression(mu))
> plot(post_step_02$sigma_chain, type = "l", col = "blue", main = "Trace of Sigma (Step = 0.02)", xlab = "Iteration", ylab = expression(sigma))
>
> par(mfrow = c(1, 1))
>
```

2.4 graphs



2.4

1. Visually inspecting the trace plot reveals a significant mixing problem specifically within the small step-size ($\text{step} = 0.001$) chains.
2. Instead of a rapidly fluctuating, stationary "furry caterpillar" pattern, the $\text{step} = 0.001$ chains snake and wander slowly over time.
3. This behavior indicates extremely high autocorrelation, meaning each proposed sample is heavily dependent on the previous state.
4. Consequently, the chain fails to efficiently explore the parameter space, resulting in poor mixing and unreliable posterior estimates.

```
Untitled1* x
Source on Save
Run
Source

1
2 # EXERCISE 2.5
3 set.seed(42)
4 post_400_5 <- HMC(y = y, n = length(y), m = 400, s = 5, a = 10, b = 2,
5                 step = 0.02, L = 12, initial_q = c(1000, 11), nsamp = 6000, nburn = 2000)
6 set.seed(42)
7 post_400_20 <- HMC(y = y, n = length(y), m = 400, s = 20, a = 10, b = 2,
8                 step = 0.02, L = 12, initial_q = c(1000, 11), nsamp = 6000, nburn = 2000)
9 set.seed(42)
10 post_1000_5 <- HMC(y = y, n = length(y), m = 1000, s = 5, a = 10, b = 2,
11                 step = 0.02, L = 12, initial_q = c(1000, 11), nsamp = 6000, nburn = 2000)
12 set.seed(42)
13 post_1000_20 <- HMC(y = y, n = length(y), m = 1000, s = 20, a = 10, b = 2,
14                 step = 0.02, L = 12, initial_q = c(1000, 11), nsamp = 6000, nburn = 2000)
15 set.seed(42)
16 post_1000_100 <- HMC(y = y, n = length(y), m = 1000, s = 100, a = 10, b = 2,
17                 step = 0.02, L = 12, initial_q = c(1000, 11), nsamp = 6000, nburn = 2000)
18 d1 <- density(post_400_5$mu_chain)
19 d2 <- density(post_400_20$mu_chain)
20 d3 <- density(post_1000_5$mu_chain)
21 d4 <- density(post_1000_20$mu_chain)
22 d5 <- density(post_1000_100$mu_chain)
23
24 xlim_all <- range(c(d1$x, d2$x, d3$x, d4$x, d5$x))
25 ylim_all <- range(c(0, d1$y, d2$y, d3$y, d4$y, d5$y))
26 plot(d5, col = "black", lwd = 2, xlim = xlim_all, ylim = ylim_all,
27      main = expression(paste("Posterior Sensitivity to Prior on ", mu)),
28      xlab = expression(mu), ylab = "Density")
29 lines(d1, col = "red", lwd = 2)
30 lines(d2, col = "orange", lwd = 2)
31 lines(d3, col = "blue", lwd = 2)
32 lines(d4, col = "purple", lwd = 2)
33
34 abline(v = 800, col = "darkgray", lty = 2, lwd = 2)
35 legend("topright",
36       legend = c("m=1000, s=100 (Baseline)", "m=400, s=5", "m=400, s=20",
37                 "m=1000, s=5", "m=1000, s=20", "True Value (800)"),
38       col = c("black", "red", "orange", "blue", "purple", "darkgray"),
39       lty = c(1, 1, 1, 1, 1, 2), lwd = 2, bty = "n", cex = 0.8)
```

24:51 (Top Level) R Script

Console

2.5 graphs

Posterior Sensitivity to Prior on μ

